

포팅 매뉴얼

포팅 매뉴얼

Frontend (Web)

- Framework : React
- Language : JavaScript (ES6 +)
- Build Tool : Vite / Create React App
- HTTP Client : Axios

Frontend (Mobile)

- Framework : Android Studio
- Language : Kotlin / Java
- Build Tool : Gradle

Backend

- **Framework** : Spring Boot 3.5.3
- **Language** : Java21
- **Build Tool** : Gradle 8.5
- **Database** : PostgreSQL
- **Cache** : Redis 7
- **Additional Dependencies**
 - Spring Data JPA
 - Spring Security
 - Spring Data Redis
 - JWT

- SpringDoc OpenAPI

AI

- **Framework** : Fast API
- **Language** : Python 3.10 +
- **ML Libraries**
 - TensorFlow / PyTorch
 - scikit-learn
 - pandas / numpy
- **Additional Tools**
 - Jupyter Notebook(모델 개발)

CI/CD

- **Jenkins** : Pipeline 기반 자동 배포
- **Docker** : 컨테이너 화
- **Docker Compose** : 서비스 오케스트레이션

2. 환경변수

.env 파일 설정

Docker Compose 환경 변수

3. 설정 파일

Backend (application.yml)

- 데이터베이스 : PostgreSQL (j13d105.p.ssafy.io:5432)
- Redis :

- **메일 서버** : Naver SMTP
- **문자 서버** : Cool SMS
- **API Keys** : SSAFY, GMS

```
# src/main/resources/application.yml

jwt:
  secret: 7tI0L6tiWxOKdSPnMupJNXglnRURuU2q-Sba1o6nD7Y_HfG8tA9y3
  fJzvVw1kLqR0bJxN4mP6QeV1R2S8T0YzU1X2W3a5B7C9
  expiration: 3600000

aes:
  key: ckOQg4AiAoTKZukU
  iv: 1234567890abcdef

ssafy:
  api:
    base-url: https://finopenapi.ssafy.io/ssafy/api/v1
    api-key: 516ac0d7d87941a78c14ff13e70770ea
    timeout: 30000
    max-retry: 3

spring:
  ai:
    openai:
      api-key: S13P22D105-98ce34b2-3d3a-458c-8116-a79c335278a9
      base-url: https://gms.ssafy.io/gmsapi/api.openai.com/v1
      chat:
        options:
          model: "gpt-4.1-mini"
          temperature: 0.3
          max-tokens: 4096
          top-p: 1.0
          frequency-penalty: 0.0
          presence-penalty: 0.0
```

retry:
 max-attempts: 3
 backoff:
 initial-interval: 2s
 multiplier: 5
 max-interval: 3m
 timeout: 60s

mail:
 host: smtp.naver.com
 port: 587
 username: vaakheejn@naver.com
 password: SNCZ4YPXKS4H
 properties:
 mail:
 smtp:
 auth: true
 starttls:
 enable: true

coolsms:
 api:
 key: NCS9ZZBLWWAI03Q7
 secret: TOVQH5Q7O1FNPETBJXMPQXIANB1WVMDD
 sms:
 sender-phone: 01093652512

management:
 health:
 mail:
 enabled: false

```

//build.gradle

plugins {
    id 'java'
    id 'org.springframework.boot' version '3.5.5'
    id 'io.spring.dependency-management' version '1.1.7'
}

group = 'com.example'
version = '0.0.1-SNAPSHOT'
description = 'Demo project for Spring Boot'

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}

configurations {
    compileOnly {
        extendsFrom annotationProcessor
    }
}

repositories {
    mavenCentral()
    maven { url 'https://repo.spring.io/milestone' }
    maven { url 'https://repo.spring.io/snapshot' }
}

dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-data-redis'
    implementation 'org.springframework.boot:spring-boot-starter-mail'
    implementation 'org.springframework.boot:spring-boot-starter-security'
    implementation 'org.springframework.boot:spring-boot-starter-web'
    compileOnly 'org.projectlombok:lombok'
}

```

```

runtimeOnly 'org.postgresql:postgresql'
annotationProcessor 'org.projectlombok:lombok'
testImplementation 'org.springframework.boot:spring-boot-starter-test'
testImplementation 'org.springframework.security:spring-security-test'
testRuntimeOnly 'org.junit.platform:junit-platform-launcher'

implementation 'org.springframework.boot:spring-boot-starter-validation'
implementation 'org.springframework.boot:spring-boot-starter-webflux'
implementation 'io.jsonwebtoken:jjwt-api:0.12.6'
runtimeOnly 'io.jsonwebtoken:jjwt-impl:0.12.6'
runtimeOnly 'io.jsonwebtoken:jjwt-jackson:0.12.6'
implementation 'com.github.ulisesbocchio:jasypt-spring-boot-starter:3.0.5'
implementation 'com.google.firebase:firebase-admin:9.4.1'
implementation 'org.springframework.boot:spring-boot-starter-quartz'
implementation 'org.springframework.boot:spring-boot-starter-actuator'
implementation 'org.bouncycastle:bcprov-jdk18on:1.79'
implementation 'org.springdoc:springdoc-openapi-starter-webmvc-ui:2.8.6'
implementation 'net.logstash.logback:logstash-logback-encoder:7.4'
implementation 'net.nurigo:sdk:4.3.0'
implementation 'org.springframework.retry:spring-retry'
implementation 'org.springframework.boot:spring-boot-starter-aop'

implementation 'org.springframework.ai:spring-ai-openai-spring-boot-starter:1.0.0-M2'

//단위테스트를 위한 의존성 추가
testImplementation 'org.springframework.boot:spring-boot-starter-test'
testImplementation 'org.junit.jupiter:junit-jupiter:5.8.2'
testImplementation 'org.mockito:mockito-core:4.6.1'
testImplementation 'org.mockito:mockito-junit-jupiter:4.6.1'
testImplementation 'org.assertj:assertj-core:3.23.1'
testImplementation 'com.fasterxml.jackson.core:jackson-databind'
}

tasks.named('test') {

```

```
useJUnitPlatform()  
}
```

Frontend

AI (FastAPI)

4. EC2 포트 설정

시스템 정보

- **OS** : Ubuntu 22.04.5 LTS (Jammy)
- **Kernel** : Linux 6.8.0-1024-aws

현재 사용 중인 포트

- **443** : HTTPS (Nginx SSL)
- **80** : HTTP (Nginx - HTTPS 리다이렉트)
- **3000** : Frontend (React) - Nginx 프록시
- **8081** : Backend (Spring Boot) - Nginx 프록시
- **8080** : Jenkins (Docker 컨테이너)
- **6379** : Redis (localhost 바인딩)
- **5432** : PostgreSQL (외부 접근)

Nginx 프록시 설정

- **Domain** : j13d105.p.ssafy.io
- **SSL** : Let's Encrypt 인증서
- **API 라우팅** :

- /api/ → Backend (8081)
- /docs, /redoc → FastAPI (8000)
- AI 추천 엔드포인트들 → FastAPI (8000)
- 나머지 → Frontend (3000)

```
# /etc/nginx/sites-available/default
```

```
# HTTP to HTTPS redirect
```

```
server {
    listen 80;
    server_name i13d208.p.ssafy.io;
    return 301 https://$server_name$request_uri;
}
```

```
# HTTPS server
```

```
server {
    listen 443 ssl;
    server_name i13d208.p.ssafy.io;
```

```
# SSL 인증서 설정
```

```
ssl_certificate /etc/letsencrypt/live/i13d208.p.ssafy.io/fullchain.pem;
ssl_certificate_key /etc/letsencrypt/live/i13d208.p.ssafy.io/privkey.pem;
```

```
# SSL 보안 설정 (권장)
```

```
ssl_protocols TLSv1.2 TLSv1.3;
ssl_ciphers HIGH:!aNULL:!MD5;
ssl_prefer_server_ciphers on;
```

```
# =====
```

```
# Spring Boot - Port 8081
```

```
# =====
```

```
# Spring Boot API
```

```
location /api/ {
    proxy_pass http://localhost:8081/api/;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
```

```

proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;

# 모든 헤더 전달 (Authorization 포함)
proxy_pass_request_headers on;
proxy_set_header Authorization $http_authorization;

# CORS 설정
add_header 'Access-Control-Allow-Origin' '*' always;
add_header 'Access-Control-Allow-Methods' 'GET, POST, PUT, DELETE, OPTIONS, PATCH' always;
add_header 'Access-Control-Allow-Headers' 'Accept, Authorization, Cache-Control, Content-Type, DNT, If-Modified-Since, Keep-Alive, Origin, User-Agent, X-Requested-With' always;

# OPTIONS 요청 처리 (preflight)
if ($request_method = OPTIONS) {
    add_header 'Access-Control-Allow-Origin' '*';
    add_header 'Access-Control-Allow-Methods' 'GET, POST, PUT, DELETE, OPTIONS, PATCH';
    add_header 'Access-Control-Allow-Headers' 'Accept, Authorization, Cache-Control, Content-Type, DNT, If-Modified-Since, Keep-Alive, Origin, User-Agent, X-Requested-With';
    add_header 'Access-Control-Max-Age' 1728000;
    add_header 'Content-Type' 'text/plain; charset=utf-8';
    add_header 'Content-Length' 0;
    return 204;
}

# Spring Boot Swagger UI
location /swagger-ui/ {
    proxy_pass http://localhost:8081/swagger-ui/;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}

```

```
location /swagger/ {
    proxy_pass http://localhost:8081/swagger/;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

```
location /v3/api-docs {
    proxy_pass http://localhost:8081/v3/api-docs;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

```
# =====
# React Frontend - Port 3000
# =====
```

```
location / {
    # GeoIP를 이용한 국가별 접속 차단
    if ($allowed_country = no) {
        return 403;
    }
}
```

```
proxy_pass http://localhost:3000;
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
```

```
# WebSocket 지원 (필요시)
proxy_http_version 1.1;
proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection "upgrade";
```

```
}  
}
```

5. 도커 설정

Docker Compose 구성

- **Redis** : redis: 7-alpine
- **Backend** : Gradle 8.5-jpk21-alpine 기반 멀티스테이지 빌드
- **Frontend** : Node 20-alpine 기반

```
#docker-compose.yml
```

```
services:
```

```
  redis:
```

```
    image: redis:7.2.4-alpine
```

```
    container_name: d105_redis
```

```
    ports:
```

```
      - "127.0.0.1:6379:6379"
```

```
    volumes:
```

```
      - ./redis-data:/data
```

```
    networks:
```

```
      - d105-net
```

```
d105-app:
```

```
  build:
```

```
    context: ./backend/d105
```

```
    dockerfile: Dockerfile
```

```
    container_name: d105-app
```

```
    ports:
```

```
      - "8081:8080"
```

```
    volumes:
```

```
      - ./d105-logs:/logs
```

```
    environment:
```

```
      - SPRING_DATASOURCE_URL=jdbc:postgresql://j13d105.p.ssafy.io:543
```

2/d105_db

- SPRING_DATASOURCE_USERNAME=d105user
- SPRING_DATASOURCE_PASSWORD=xmrghkd105!!
- SPRING_DATA_REDIS_HOST=d105_redis
- SPRING_DATA_REDIS_PORT=6379

depends_on:

- redis

networks:

- d105-net

d105-frontend:

build:

context: ./frontend/react/D105

dockerfile: Dockerfile

container_name: d105-frontend

ports:

- "3001:3000"

depends_on:

- d105-app

networks:

- d105-net

environment:

- VITE_API_URL=http://j13d105.p.ssafy.io:8081
- REACT_APP_API_URL=http://j13d105.p.ssafy.io:8081

loki:

image: grafana/loki:2.9.4

container_name: loki

ports:

- "127.0.0.1:3100:3100"

command: ["-config.file=/etc/loki/local-config.yaml"]

networks:

- d105-net

grafana:

image: grafana/grafana:10.4.2

container_name: grafana

ports:

- "3000:3000"

```

volumes:
  - ./grafana-data:/var/lib/grafana
environment:
  - GF_SECURITY_ADMIN_PASSWORD=admin
user: "0"
depends_on:
  - loki
networks:
  - d105-net

promtail:
  image: grafana/promtail:2.9.4
  container_name: promtail
  volumes:
    - /var/jenkins_home/workspace/d105/promtail-config.yml:/tmp/promtail
    -config.yml:ro
    - /var/jenkins_home/workspace/d105/d105-logs:/logs:ro
  command: ["-config.file=/tmp/promtail-config.yml"]
  depends_on:
    - d105-app
    - loki
  networks:
    - d105-net

networks:
  d105-net:
    driver: bridge

volumes:
  grafana-data:

```

```
#backend/d105/Dockerfile
```

```
FROM gradle:8.5-jdk21-alpine AS build
```

```
WORKDIR /app
```

```
# 전체 프로젝트 복사
```

COPY . .

Gradle Wrapper 권한 설정 (있다면)

RUN chmod +x gradlew || true

애플리케이션 빌드

RUN gradle clean build --no-daemon -x test

실행 단계

FROM openjdk:21-jdk-slim

WORKDIR /app

타임존 설정

RUN apt-get update && apt-get install -y tzdata

ENV TZ=Asia/Seoul

RUN ln -snf /usr/share/zoneinfo/\$TZ /etc/localtime && echo \$TZ > /etc/timezone

빌드된 JAR 파일 복사 (d105 프로젝트명으로 수정)

COPY --from=build /app/build/libs/d105-0.0.1-SNAPSHOT.jar app.jar

EXPOSE 8080

CMD ["java", "-Duser.timezone=Asia/Seoul", "-jar", "app.jar"]

#Frontend/React/d105/Dockerfile

FROM node:20-alpine

WORKDIR /app

package.json과 package-lock.json 복사

COPY package*.json ./

의존성 설치

RUN npm install

소스 코드 복사

```
COPY ..
```

```
# 빌드 실행
```

```
RUN npm run build
```

```
# serve 글로벌 설치
```

```
RUN npm install -g serve
```

```
# 3000 포트 노출
```

```
EXPOSE 3000
```

```
# serve로 dist 폴더 서빙 (3000 포트)
```

```
CMD ["serve", "-s", "dist", "-l", "3000"]
```

6. 젠킨스 설치

현재 Jenkins 설정

- 버전 : jenkins/jenkins:lts (Docker 컨테이너)
- 포트 : 8080 (외부 접근 가능)
- 실행 방법 : Docker 컨테이너 실행 중
- 볼륨 : jenkins_data

Jenkin 설치 방법

```
# Jenkins
```

```
를
```

```
Docker
```

```
로
```

```
설치
```

```
docker run -d \--name jenkins \-p 8080:8080 \-p 50000:50000 \-v jenkins_
data:/var/jenkins_home \-v /var/run/docker.sock:/var/run/docker.sock \
```

```
포팅
```

매뉴얼

23

jenkins/jenkins:its

Jenkins

에

Docker

권한

부여

```
docker exec -u 0 jenkins apt-get update
```

```
docker exec -u 0 jenkins apt-get install -y docker.io
```

```
docker exec -u 0 jenkins usermod -aG docker jenkins
```

```
docker restart jenkins
```

```
#Jnekinsfile
```

```
pipeline {
```

```
  agent any
```

```
  stages {
```

```
    stage('Cleanup') {
```

```
      steps {
```

```
        echo 'Stopping existing containers...'
```

```
        sh 'docker-compose down || true'
```

```
        sh 'docker image prune -f || true'
```

```
      }
```

```
    }
```

```
    stage('Prepare Config Files') {
```

```
      steps {
```

```
        echo 'Ensuring config files exist as files, not directories...'
```

```
        sh '''
```

```
          # 디렉토리로 생성된 경우 제거
```

```
          [ -d promtail-config.yml ] && rm -rf promtail-config.yml
```

```
          # Git에서 파일 체크아웃
```

```
          git checkout HEAD -- promtail-config.yml || echo "promtail-config.yml not in git"
```

```

        ls -la promtail-config.yml
        cat promtail-config.yml
    ""
}

stage('Build and Deploy') {
    steps {
        echo 'Building and deploying with Docker Compose...'
        sh 'docker-compose up -d --build'
    }
}

stage('Health Check') {
    steps {
        echo 'Waiting for application to start...'
        sleep(time: 30, unit: 'SECONDS')
        echo 'Checking application health...'
        sh 'curl -f http://localhost:8081/actuator/health || echo "App not ready yet"'
    }
}

stage('Monitoring Check') {
    steps {
        echo 'Checking monitoring services...'
        sleep(time: 10, unit: 'SECONDS')
        echo 'Checking container status...'
        sh 'docker-compose ps'
        echo 'Checking Prometheus logs...'
        sh 'docker logs prometheus || echo "Prometheus container not running"'
        echo 'Testing Grafana...'
        sh 'curl -f http://localhost:3000/api/health || echo "Grafana not ready yet"'
    }
}

```

```
}  
}
```

Jenkins Pipeline 설정

- 프로젝트명 : spring-react-app
- 트리거 : GitLab 웹훅 (Push events, Merge request events)
- 빌드 단계
 1. Checkout (GitLab)
 2. Build Frontend (npm install & build)
 3. Build Backend (Gradle clean build)
 4. Stop Current Service
 5. Deploy (Docker Compose)
 6. Health Check

GitLab 웹훅 설정

웹훅 URL : <https://lab.ssafy.com/s13-fintech-finance-sub1/S13P21D105.git>

트리거 : Push events, Merge request events

시크릿 토큰 : Jenkins에서 생성된 토큰 사용

Jenkins 권한 설정

```
#Jenkins 컨테이너에 Docker 접근 권한 부여
docker exec -u 0 jenkins apt-get update
docker exec -u 0 jenkins apt-get install -y docker.io
docker exec -u 0 jenkins usermod -aG docker jenkins
#Jenkins 재시작
docker restart jenkins
```

7. 추가 인프라 설정

PostgreSQL 설치 및 설정

```
# 패키지 목록 업데이트
sudo apt update

# PostgreSQL 설치 (최신 버전)
sudo apt install postgresql postgresql-contrib -y
```

Nginx 설치 및 SSL 설정

```
#Nginx 설치
sudo apt install nginx

#Let's Encrypt 인증서 설치
sudo apt install certbot python3-certbot-nginx

#SSL 인증서 발급
sudo certbot --nginx -d i13d208.p.ssafy.io
```

